

# VisionEngine

ユーザーのように**UI**を見る——コンピュータビジョンと**LLM**ビジョンによる分析とナビゲーション

## 概要

VisionEngineは、従来のコンピュータビジョンとLLMベースのビジョンを組み合わせた、分離型のGoツールキットです。ユーザーインターフェースの分析、UI要素や視覚的な問題の検出、アプリ画面遷移のナビゲーショングラフ構築を可能にします。複数のビジョンプロバイダーに対応し、OpenCVはビルドタグによって制御されています。

## 短い説明

UI分析とナビゲーショングラフ構築のための再利用可能なGoモジュールです。アナライザー層（UI要素、画面差分、視覚的問題）、BFS経路探索とDOT/JSON/Mermaidエクスポート機能を持つナビゲーショングラフ、そしてGPT-4o、Claude、Gemini、Qwen-VLなどに対応したLLMビジョンアダプターを提供します。

## 長い説明

ほとんどのUIテスト自動化は、事実上「盲目」です。アクセシビリティツリーやDOMセレクターに頼ることで、機械が認識するインターフェースを操作しているに過ぎず、人間が実際に体験する部分——ボタンが正しく表示されているか、レイアウトが崩れていないか、遷移先の画面が期待通りか——を見落としていきます。VisionEngineは、このギャップを埋めることで、自動化に真の「知覚」を与えます。つまり、人間と同じようにUIを見て、推論する能力を持たせるのです。このツールキットは、生のピクセルデータからアプリ全体の理解に至るまで、4つの協調するレイヤーで構成されています。

アナライザーは、安定した契約を定義します。インターフェース（Analyzer、VideoProcessor）や値型（UIElement、ScreenAnalysis、ScreenDiff、Rect、Size、TextRegion、VisualIssue、ScreenIdentity、Action、KeyFrame）を持ち、StubAnalyzerというリファレンス実装も提供

されています。これにより、利用者は要素の検出、画面の差分比較、視覚的な問題の発見を、契約が変わることのない安定した基盤の上で行えます。

ナビゲーショングラフは、単一の画面からアプリ全体へと視野を広げ、画面遷移を有向グラフとしてモデル化します。BFSによる経路探索と、3つのエクスポートバックエンド（DOT、JSON、Mermaid）を備えており、自動化は画面を見るだけでなく、任意の画面へのルートを計画できるようになります。さらに、ストレステスト、自動化テスト、統合テスト、セキュリティテストのスイートも提供され、その有効性が実証されています。

**LLM**ビジョンレイヤーは、最新のマルチモーダル推論を追加します。VisionProviderインターフェースには、OpenAI（GPT-4o）、Anthropic（Claude）、Gemini、Qwen-VL、Kimi、StepGUI、Astica、Ollamaなどのアダプターが用意されており、FallbackChainによって構成されています。これにより、特定のプロバイダーが失敗したり、レート制限に達したり、性能が不十分な場合でも、次のプロバイダーに自動的にフォールバックし、テスト全体が停止することを防ぎます。

設定レイヤーは、環境変数の読み込みと検証を担当し、ユーザー向けのエラーメッセージはすべてi18n.Translatorを通じて出力されます。

このツールキットを実際に採用可能にしているのは、重いネイティブ依存がオプションである点です。OpenCVのバインディングは-tags visionというビルドタグによって制御されており、デフォルトのビルドではスタブが提供されます。そのため、Go 1.25以降のホストであれば、OpenCVのツールチェーンがなくてもモジュール全体がコンパイル、テスト、実行可能です。ネイティブスタックは、利用者が明示的に選択した場合にのみ組み込まれます。これにより、VisionEngineは特別なイメージを必要とせず、通常のCIランナーにも簡単に導入できます。

完全に分離された設計（CONST-051(B)に準拠）により、VisionEngineは他のコードベースと同等のサブモジュールとして組み込まれます。特にHelixQAでは、エビデンスに基づくUIテストに「本物の目」を提供しています。

コンテンツ

## なぜ開発したのか

アクセシビリティツリーやセレクトタにのみ依存するUIテスト自動化では、ユーザーが実際に目にするものを見逃してしまいます。VisionEngineは、要素検出、画面差分比較、LLMビジョンによる推論といった本物の視覚理解を実現し、さらにアプリ画面のナビゲーション可能なマップを提供することで、自動化がUIを認識し、ルーティングできるようにします。

# なぜゲームチェンジャーなのか

通常は相容れない2つのアプローチ——高速で決定論的な従来型コンピュータビジョンと、柔軟で意味理解に優れたLLMビジョン——を単一のインターフェースで統合し、フォールバックチェーンを備えることで、ユーザーは一方の精度ともう一方の推論能力を選択することなく両方を手にできます。さらにOpenCVを厳密にオプション扱いにすることで、そのパワーを得るための従来の負担を排除し、どんなGoプロジェクトでも、ネイティブなビジョンツールチェーンをビルドに組み込むことなく、本物のUI認識を実現できます。

## 革新的なポイント

- デュアル認識：従来型CV（OpenCV/GoCV）とマルチプロバイダーLLMビジョンをフォールバックチェーンで統合。
- ナビゲーショングラフ：BFS経路探索とDOT/JSON/Mermaid形式でのエクスポートに対応。
- ビルドタグによる**OpenCV**の制御：ネイティブ依存なしでもモジュールのビルドとテストが可能。
- 完全に分離された国際化対応サブモジュール：HelixQAでも使用される同一コードベースの独立モジュール。

## 課題と解決策

- ネイティブ依存の重さ：-tags visionによる制御とデフォルトスタブで解決。OpenCVがなくてもCI/ホスト環境でビルドとテストが可能。
- ビジョンプロバイダーの信頼性低下：VisionProviderインターフェースとFallbackChainコンポーザーで解決。
- 複雑なアプリフローのマッピング：有向ナビゲーショングラフとBFS経路探索、複数形式でのエクスポートで解決。
- 結合度の高さ：CONST-051(B)による分離とi18nトランスレータシームで解決。

## 技術スタック（なぜ+どのように）

- **Go (1.25+)**：モジュールコアおよび4つのレイヤー。
- **GoCV / OpenCV**：従来型コンピュータビジョン。ビルドタグによる制御。
- **LLMビジョンプロバイダー（GPT-4o、Claude、Gemini、Qwen-VL、Kimi、StepGUI、Astica、Ollama）**：アダプターを介したマルチモーダルUI推論。
- **グラフアルゴリズム（BFS）**：ナビゲーション経路探索。
- **DOT / JSON / Mermaid**エクスポーター：ナビゲーショングラフの可視化。
- **i18n**トランスレータ：分離されたユーザー向け文字列。